SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK**
Weightage:	**20%**	**Total Marks:**	**25**
CO2 Statement:	*Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)*		
Student Name:	M. HARINI	**Reg.No.:**	192424011
Department:	AI&DS	**Date:**	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I M.HARINI (192424011) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: M.HARINI

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	**Debugging Approach & Analysis (20%)**	**Correctness of Debugged Solution (25%)**	**Optimization Techniques Applied (20%)**	**Testing and Documentation (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

1. Debugging Signed Integer Addition Using 2's Complement

Problem : A program performing signed integer addition using **8-bit 2's complement** produces incorrect results when adding **+120** and **+50**. The output becomes a negative number instead of the expected positive value.

Step 1: Convert Decimal Numbers to Binary

+120	**+50**
120 = 01111000	50 = 00110010

Step 2: Perform Binary Addition

```
   01111000   (+120)
+  00110010   (+50)
   ----------
   10101010
```

The binary result is: 10101010

Since the MSB (Most Significant Bit) is **1**, the processor interprets this number as negative.

Finding its decimal value: Invert bits

10101010
↓

01010101

Add 1

01010110 = 86

Result = -86

The processor gives **−86**, but mathematically
120 + 50 = 170

Step 3: Why is the Result Wrong?

An **8-bit signed integer** can represent only

Minimum = -128 Maximum = +127

The actual answer is 170

which is outside the valid range. Therefore, **overflow has occurred**.

Step 4: Overflow Detection

Overflow in 2's complement occurs when

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

Here,
+120 → Positive
+50 → Positive

Result = -86 (Negative)

Hence Overflow = YES

Another method is checking carry bits. Carry into sign bit ≠ Carry out of sign bit

Overflow = 1

Step 5: Debugging the Problem

Possible reasons include:

- Overflow flag not checked.
- Programmer assumed every addition fits into 8 bits.
- Incorrect data type selected.
- Arithmetic performed without overflow detection.

Step 6: Proposed Solution

Method 1 – Detect Overflow

After every signed addition,
if (Overflow == 1)
{
  Display "Arithmetic Overflow";
}

Method 2 – Use Larger Data Type

Instead of 8-bit integers,16-bit integer

Range:
-32768 to +32767 So, 170 fits easily.

Method 3 – Saturation Arithmetic

Instead of wrapping around,

If result > 127 Store 127 or If result < -128 Store -128

This method is commonly used in DSP processors.

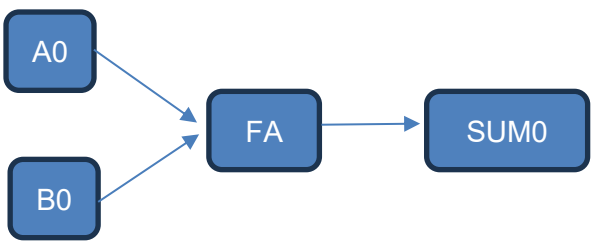
Conclusion

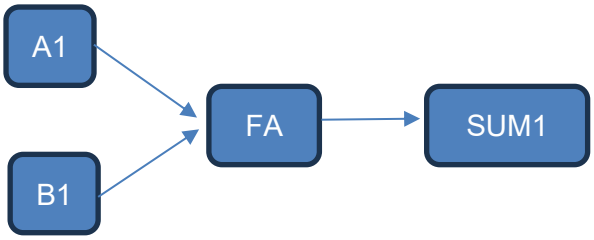
The incorrect output occurs because **170 cannot be represented in an 8-bit signed integer**. The processor wraps around to **−86**, causing overflow. This can be corrected by checking the overflow flag, using a larger integer size, or applying saturation arithmetic.

2. Debugging Ripple Carry Adder Performance

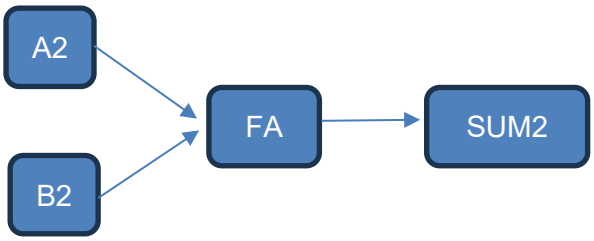
Problem : A processor uses a **Ripple Carry Adder (RCA)**. Arithmetic operations take too much time because every carry must travel through all previous full adders.

Ripple Carry Adder Structure



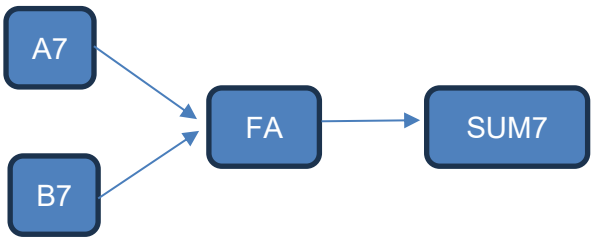


Carry



Carry

.
.
.



Each Full Adder waits for the previous carry.

Carry Propagation

Suppose

11111111
+ 00000001

Carry moves as

Bit0 → Bit1 → Bit2 → Bit3 → Bit4 → Bit5 → Bit6 → Bit7

Every stage must wait.

Total Delay = Number of bits × Delay of one Full Adder

For an 8-bit adder

Delay = 8FA

For a 32-bit adder

Delay = 32FA

This becomes a bottleneck in real-time systems.

Debugging the Bottleneck

The delay occurs because

- Carry is generated sequentially.
- No carry prediction.
- Every Full Adder depends on the previous one.

Optimized Solution – Carry Look-Ahead Adder (CLA)

Instead of waiting, CLA predicts carries.

It computes
Generate

$$G = A \times B$$

Propagate

$$P = A \oplus B$$

Carry equations

$$C1 = G0 + P0C0$$

$$C2 = G1 + P1G0 + P1P0C0$$

$$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$$

All carries are computed simultaneously.

Comparison

Ripple Carry Adder	Carry Look-Ahead Adder
Sequential carry	Parallel carry
Slow	Fast
Large delay	Small delay
Simple hardware	More hardware
Low cost	Higher cost

Advantages
- Faster execution
- Reduced propagation delay
- Suitable for high-speed processors
- Better real-time performance

Conclusion
The delay is caused by sequential carry propagation. Replacing the Ripple Carry Adder with a Carry Look-Ahead Adder computes carries in parallel and significantly increases processing speed.

3. Debugging Booth's Multiplication Algorithm

Problem : A multiplication unit implementing **Booth's Algorithm** gives incorrect results when multiplying negative numbers.

Booth's Algorithm Registers

$$M \rightarrow \text{Multiplicand}$$

$$Q \rightarrow \text{Multiplier}$$

$$A \rightarrow \text{Accumulator}$$

$$Q\text{-}1 \rightarrow \text{Extra Bit}$$

Decision Table

Q0	Q-1	Operation
00		No operation
01		Add M
10		Subtract M
11		No operation

Example
Multiply
+5 × -3
Binary
M = 0101

Q = 1101
Each cycle

1. Check Q0 and Q-1
2. Perform Add/Subtract
3. Arithmetic Right Shift

Repeat.

Debugging

Possible errors include
Error 1
Wrong bit pair checking.
Example
01 treated as 10
Wrong operation performed.

Error 2
Logical Right Shift
Before

11101010
↓
01110101

Sign changes incorrectly.

Correct Shift
Arithmetic Right Shift

11101010
↓
11110101

Sign remains correct.

Error 3
Incorrect Sign Extension
Negative numbers must preserve the sign bit.

Error 4
Incorrect 2's Complement
Subtraction should use
A = A + (2's complement of M)
Any mistake changes the result.

Corrections

- Check Q0 and Q-1 correctly.
- Use Arithmetic Right Shift.
- Preserve sign bit.
- Verify subtraction logic.
- Test with positive and negative operands.

Conclusion
Incorrect signed multiplication is mainly caused by improper bit-pair checking, logical shifting instead of arithmetic shifting, or incorrect sign extension. Correct implementation of Booth's Algorithm ensures accurate signed multiplication.

4. Debugging Restoring Division Algorithm

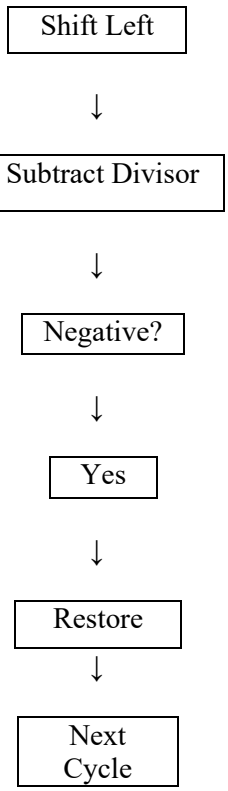
Problem : An embedded processor using the **Restoring Division Algorithm** takes excessive execution time because restoration occurs frequently.

Restoring Division Steps
1. Shift left.
2. Subtract divisor.
3. Check remainder.
4. If remainder is negative

Restore previous value

5. Repeat.

Workflow



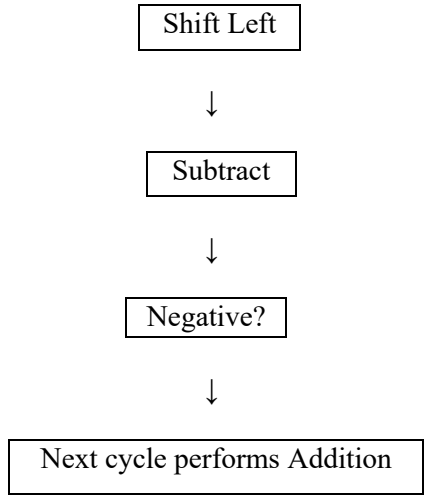
Problem
Whenever subtraction becomes negative,The processor performs another addition.
Extra operations increase execution time.

Debugging
Main causes
- Frequent restoration
- More ALU operations
- Higher power consumption
- Increased clock cycles

Optimized Solution
Use

Non-Restoring Division

Algorithm



instead of Restoration
Restoration is delayed.

Comparison

Restoring	Non-Restoring
Frequent restoration	No immediate restoration
More operations	Fewer operations
Slower	Faster
Higher execution time	Lower execution time

Advantages

- Reduced arithmetic operations
- Lower processor delay
- Improved throughput
- Better embedded system performance

Conclusion

Replacing the Restoring Division Algorithm with the Non-Restoring Division Algorithm removes unnecessary restoration operations, reducing execution time and improving overall processor efficiency.

5. Debugging IEEE 754 Floating-Point Addition

Problem : A scientific application uses **IEEE 754 Single Precision Floating Point**. When adding a very small number to a very large number, the result loses accuracy.

IEEE 754 Single Precision Format

1 Bit → Sign

8 Bits → Exponent

23 Bits → Mantissa

Example
1.0×10^{20} +
1.0
Actual Answer 10000000000000000001
Processor Result 10000000000000000000
The smaller value disappears.

Why Does This Happen?

Step 1
Align Exponents

Large exponent : 20
Small exponent : 0

The mantissa of the smaller number shifts right many places.

Step 2
Normalization
Floating-point numbers must remain normalized.
Extra bits may be discarded.

Step 3
Rounding
Only 23 mantissa bits are stored.
Remaining bits are rounded.
Small values are lost.

Sources of Error

- Exponent alignment
- Limited mantissa precision
- Normalization
- Rounding
- Loss of significance

Debugging

Check

- Exponent comparison
- Mantissa shifting
- Guard bits
- Round bits
- Sticky bits

Optimized Solutions

Solution 1
Use Double Precision

64-bit floating point

1 Sign Bit

11 Exponent Bits

52 Mantissa Bits

Much higher precision.

Solution 2
Kahan Summation Algorithm
Stores lost precision during repeated additions.
Improves numerical accuracy.

Solution 3
Add Smaller Numbers First
Instead of
Large + Small
use
Small numbers together first

then add the large number
This minimizes precision loss.

Solution 4
Avoid Unnecessary Rounding
Round only after completing the final computation whenever possible.

Conclusion

The loss of accuracy occurs because exponent alignment shifts out significant bits of the smaller number, and single precision has limited mantissa size. Using **double precision**, **Kahan Summation**, careful ordering of operations, and minimizing rounding significantly improves floating-point accuracy in scientific applications.